

# Фреймворк Bootstrap

## Фреймворк Bootstrap

---

Для вёрстки, как и для большинства других областей разработки, есть фреймворки, облегчающие решение стандартных задач.

Одно из популярных решений — фреймворк **Bootstrap**. Его идея проста:

- Bootstrap предоставляет разработчику файл с готовыми CSS-стилями;
- разработчик подключает этот CSS-файл к своим HTML-страницам;
- в HTML-тегах своих страниц разработчик указывает определённые классы, предустановленные в Bootstrap (их описание и назначение есть в документации), — и в результате элемент страницы приобретает презентабельный вид.

# Фреймворк Bootstrap

---

При работе с Bootstrap основная задача — найти подходящий стиль оформления элемента, перенести его в свою вёрстку и, возможно, чуть-чуть подправить.

CSS заготовки можно найти в специальном разделе на сайте Bootstrap:

<https://getbootstrap.com/docs/5.1/examples/>

## История Bootstrap

---

В 2010 году в недрах компании Twitter появился проект **Twitter Blueprint**. Его задачей было создание системы для разработки новых компонентов внутри компании.

**Twitter Blueprint** имел набор базовых блоков с готовым дизайном, который использовался в компании.

Это решение обрело популярность внутри компании благодаря своей простоте, так как любой сотрудник, имея минимальный опыт разработки, мог сделать новый концепт блока и предложить его для дальнейшей разработки.



# История Bootstrap

---

В этом же году на **Twitter Blueprint** проект обрёл первую популярность.

Такое решение не могло оставаться внутренней разработкой и уже в 2011 году проект вышел в открытый доступ под названием **Bootstrap**.

С тех пор Bootstrap удерживает лидерство среди инструментов по созданию сайтов. По разным статистикам, фреймворк используют от 17% до 30% всех сайтов в интернете.

Код Bootstrap можно найти на сайтах таких проектов, как:

# История Bootstrap

---

Код Bootstrap можно найти на сайтах таких проектов, как:

- [netflix.com](https://www.netflix.com) — **Netflix**, крупнейший в мире онлайн-поставщик сериалов и фильмов
- [templates.openoffice.org](https://templates.openoffice.org) — **OpenOffice**, портал шаблонов документов OpenOffice
- [gitlab.com](https://gitlab.com) — **GitLab**, один из крупнейших в мире репозиториев программного кода
- [toyota.com](https://toyota.com) — **Toyota Motor**, крупнейшая в мире автомобильная компания
- [volkswagenag.com](https://volkswagenag.com) — **Volkswagen Group**, вторая по размеру после Toyota автомобильная компания в мире
- [ford.com](https://ford.com) — **Ford Motor**, третий на рынке США производитель автомобилей после GM и Toyota и второй в Европе после Volkswagen

# Подключение Bootstrap

---

**Способ 1.** Скачать файл *bootstrap.css*, сохранить и подключить его из локальной директории:

HTML-код:

```
<head>
...
<link rel="stylesheet" href="css/bootstrap.css">
...
</head>
```

Ссылка <https://getbootstrap.com/docs/5.0/getting-started/download/>

# Подключение Bootstrap

---

**Способ 2.** Подключить CSS-файл, расположенный на сервере

Bootstrap:

HTML-код:

```
<head>
...
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/
css/bootstrap.min.css"
rel="stylesheet"

integrity="sha384-EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwF
spd3yD65VohhpuuCOmLASjC"
crossorigin="anonymous">
...
</head>
```



# Собственные стили

---

При использовании Bootstrap разработчик может подключать к страницам и собственные стили. Порядок подключения стилей:

- стили Bootstrap,
- собственный файл со стилями;
- стили в теге **<style>**;
- стили в атрибуте **style** тегов.

При конфликте стилей в приоритете будут те правила, которые описаны последними.

## Пример использования

---

Сверстаем страницу с описанием небоскреба.

*HTML-код:*

```
<div>
  <h5>Бурдж-Халифа</h5>
  <div>
    <h6>
      Дубай, Объединенные Арабские Эмираты
    </h6>
    <p> Небоскрёб высотой 828 метров, ...</p>
    <p> Строительство небоскрёба началось в 2004 году...</p>
    <a href="#">Читать целиком</a>
  </div>
</div>
```

# Пример использования

---

*Результат:*

## Бурдж-Халифа

Дубай, Объединенные Арабские Эмираты

Небоскрёб высотой 828 метров, самое высокое сооружение в мире. Форма здания напоминает сталагмит.

Строительство небоскрёба началось в 2004 году и шло со скоростью 1—2 этажа в неделю. Ежедневно на строительстве работало до 12 000 рабочих. На его создание ушло около 320 тыс. м<sup>3</sup> бетона и более 60 тыс. тонн стальной арматуры.

[Читать целиком](#)

# Пример использования

---

Подключим файл Bootstrap и добавим в классы из фреймворка.

*HTML-код:*

```
<div class="card col-6">
  <h5 class="card-header">Бурдж-Халифа</h5>
  <div class="card-body">
    <h6 class="card-subtitle">
      Дубай, Объединенные Арабские Эмираты
    </h6>
    <p> Небоскрёб высотой 828 метров, ...</p>
    <p> Строительство небоскрёба началось в 2004 году...</p>
    <a href="" class="btn btn-primary">Читать целиком</a>
  </div>
</div>
```

# Пример использования

---

*Результат:*

**Бурдж-Халифа**

**Дубай, Объединенные Арабские Эмираты**  
Небоскрёб высотой 828 метров, самое высокое сооружение в мире. Форма здания напоминает сталагмит.

Строительство небоскрёба началось в 2004 году и шло со скоростью 1—2 этажа в неделю. Ежедневно на строительстве работало до 12 000 рабочих. На его создание ушло около 320 тыс. м<sup>3</sup> бетона и более 60 тыс. тонн стальной арматуры.

Читать целиком

## Макет

---

**Контрольные точки** - это настраиваемая ширина, которая определяет поведение адаптивного макета на разных устройствах или размерах области просмотра в Bootstrap.

### Основные концепции:

- Контрольные точки - это строительные блоки адаптивного дизайна.
- По контрольным точкам необходимо описывать собственные медиа-запросы.
- Макеты ориентированы прежде всего на мобильные устройства. CSS Bootstrap нацелен на применение минимального количества стилей, чтобы макет работал с минимальной контрольной точкой, а затем настраивает этот дизайн для более крупных устройств. Это оптимизирует CSS, сокращает время рендеринга.



# Контрольные точки

Bootstrap включает шесть контрольных точек по умолчанию:

| Контрольная точка | Описание        | Идентификатор класса | Размеры |
|-------------------|-----------------|----------------------|---------|
| X-Small           | Очень маленький | None                 | <576px  |
| Small             | Маленький       | sm                   | ≥576px  |
| Medium            | Средний         | md                   | ≥768px  |
| Large             | Большой         | lg                   | ≥992px  |
| Extra large       | Очень большой   | xl                   | ≥1200px |
| Extra extra large | Огромный        | xxl                  | ≥1400px |

# Сетка Bootstrap

**Пример.** Создадим в коде два блока div и покрасим их в разные цвета.

HTML-код:

```
<div style="background: green;">Первый блок</div>  
<div style="background: yellow;">Второй блок</div>
```

Результат:



# Сетка Bootstrap

---

Bootstrap позволяет создавать из блоков колонки и задавать им необходимую ширину.

Дополним код примера так, чтобы блоки расположились в одну строку. Для этого:

- обернём цветные блоки в **div** с классом **row**, в Bootstrap полагается объединять колонки в общий блок;
- добавим к цветным блокам классы **col-3** и **col-9**, теперь они превратятся в колонки.

# Сетка Bootstrap

---

*HTML-код:*

```
<div class="row">
  <div class="col-3" style="background: green;">
    Первый блок
  </div>
  <div class="col-9" style="background: yellow;">
    Второй блок
  </div>
</div>
```

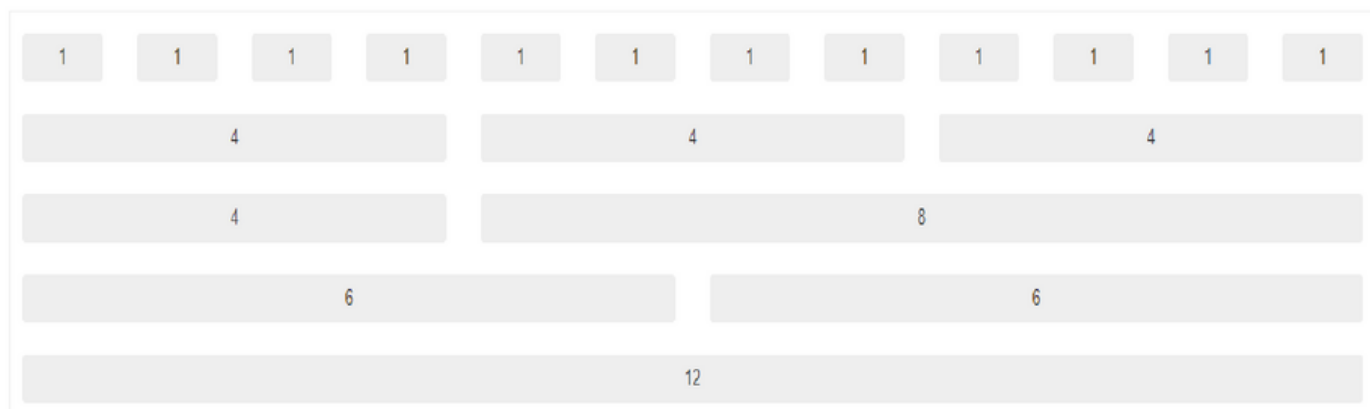
*Результат:*



# Сетка Bootstrap

---

Bootstrap работает с двенадцати колоночной сеткой.



# Сетка Bootstrap

---

Двенадцати колоночная верстка означает, что любому блоку можно установить ширину, кратную 1/12 ширины родительского блока.

Количество колонок, вложенных в `<div class="row">`, может быть от одной (она займёт всю ширину родительского блока, для этого надо установить ей класс **col-12**) до двенадцати (внутри блока с классом **row** можно создать двенадцать блоков с классом **col-1**).

Колонки можно комбинировать как угодно, лишь бы в сумме ширина колонок составляла 12.

# Сетка Bootstrap

---

В нашем примере названии класса префикс **col** означает «колонка», а цифры 3 и 9 определяют ширину колонок: это 3/12 и 9/12 от общей ширины страницы.

*HTML-код:*

```
<div class="row">
  <div class="col-3" style="background: green;">
    Первый блок
  </div>
  <div class="col-9" style="background: yellow;">
    Второй блок
  </div>
</div>
```

# Сетка Bootstrap

---

**Пример.** Создадим несколько колонок разной ширины.

*HTML-код:*

```
<div class="row">
  <div class="col-2" style="background: green;">
    Первая колонка
  </div>
  <div class="col-4" style="background: yellow;">
    Вторая колонка
  </div>
  <div class="col-6" style="background: red;">
    Третья колонка
  </div>
</div>
```



# Сетка Bootstrap

---

Результат:

| 2/12           | 4/12           | 6/12           |
|----------------|----------------|----------------|
| Первая колонка | Вторая колонка | Третья колонка |

# Сетка Bootstrap

---

Если суммарная ширина колонок превысит 12/12, лишние колонки будут перенесены на новую строку:

HTML-код:

```
<div class="row">
  <div class="col-3" style="background: green;">
    Первая колонка</div>
  <div class="col-4" style="background: yellow;">
    Вторая колонка</div>
  <div class="col-5" style="background: red;">
    Третья колонка</div>
  <div class="col-6" style="background: silver;">
    Лишняя колонка</div>
  <!-- 3/12 + 4/12 + 5/12 + 6/12 >12/12 -->
</div>
```

# Сетка Bootstrap

---

Результат:

| 3/12           | 4/12           | 5/12           |
|----------------|----------------|----------------|
| Первая колонка | Вторая колонка | Третья колонка |
| Лишняя колонка | 6/12           |                |

# Сетка Bootstrap

---

Для всех колонок можно указать просто класс **col**. Тогда колонки пропорционально разделят ширину родительского блока.

HTML-код:

```
<div class="row">
  <div class="col" style="background: green;">
    Первая колонка
  </div>
  <div class="col" style="background: yellow;">
    Вторая колонка
  </div>
  <div class="col" style="background: red;">
    Третья колонка
  </div>
</div>
```

# Сетка Bootstrap

---

Результат:

4/12

4/12

4/12

Первая колонка

Вторая колонка

Третья колонка

## Адаптивность Bootstrap

---

Для создания адаптивных страниц имена классов для именования колонок в Bootstrap построены по принципу:

**.col-[размер дисплея]-значение**

Для каждого диапазона размеров экрана предусмотрено своё сокращённое обозначение:

**.col-** – сверхмалые (<576px)

**.col-sm-** – малые (≥576px)

**.col-md-** – средние (≥768px)

**.col-lg-** – большие (≥992px)

**.col-xl-** – очень большой (≥1200px)

**.col-xxl-** – очень очень большой (≥1400)

# Адаптивность Bootstrap

---

**Пример.** Расположим элементы страницы по следующему правилу:

- на больших экранах элементы должны располагаться в четыре колонки;
- на планшетах — в две;
- на телефонах — в одну колонку.

HTML-код:

```
<div class="row">
  <div class="col-12 col-md-6 col-xl-3">Первый элемент</div>
  <div class="col-12 col-md-6 col-xl-3">Второй элемент</div>
  <div class="col-12 col-md-6 col-xl-3">Третий элемент</div>
  <div class="col-12 col-md-6 col-xl-3">Четв. элемент</div>
  <div class="col-12 col-md-6 col-xl-3">Пятый элемент</div>
  <div class="col-12 col-md-6 col-xl-3">Шестой элемент</div>
</div>
```

# Адаптивность Bootstrap

---

Каждой колонке в коде назначены три класса для определения её ширины (имена классов разделяются пробелами):

**.col-12:** этот класс — для экранов от 1px в ширину и более. Блок с классом **col-12** будет занимать всю ширину экрана (12 колонок) ( $12 / 12 = 1$ ). Если бы не было указано других классов, этот класс работал бы на дисплеях любого размера.

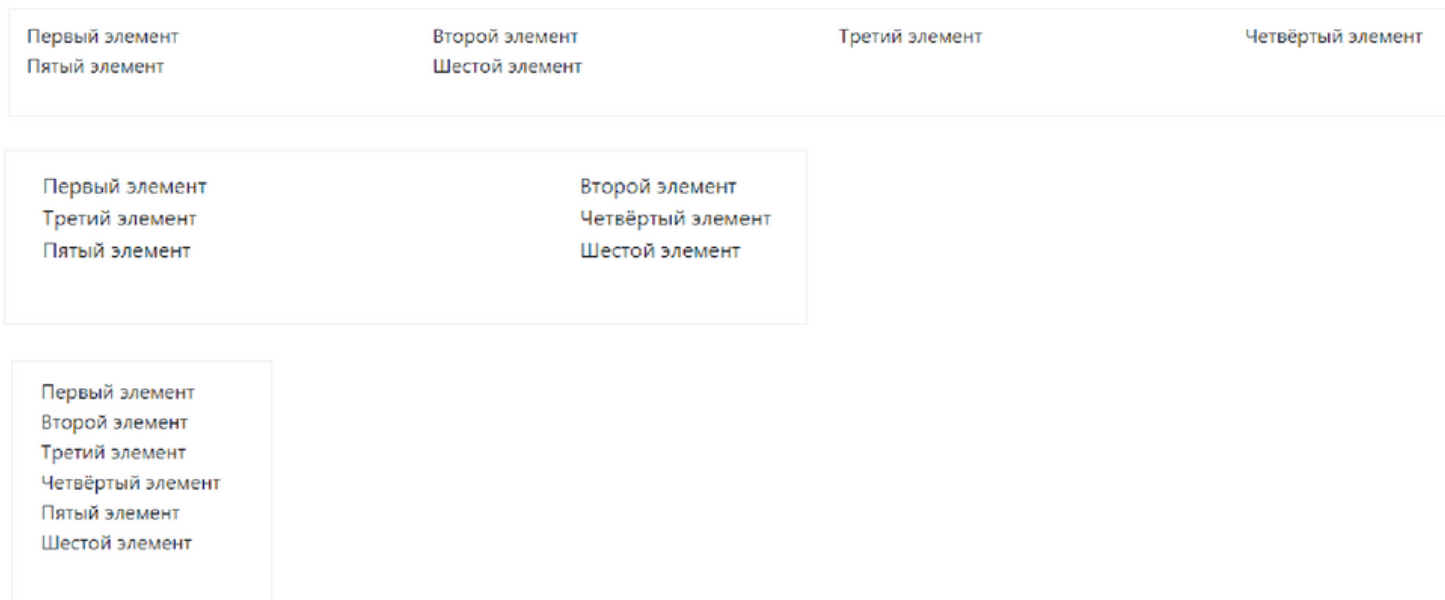
**.col-md-6:** на экранах шириной **md** (это размер экранов более 768px по горизонтали) блок будет занимать половину ширины экрана, значит, в строку уместится две такие колонки ( $12 / 6 = 2$ ).

**.col-xl-3:** на больших мониторах (размером более 1200px по горизонтали) эта колонка займёт 3 колонки из 12 доступных, значит, в строку уместится четыре такие колонки ( $12 / 3 = 4$ ).

# Адаптивность Bootstrap

---

*Результат:*



# Адаптивность Bootstrap

---

Чтобы адаптивность работала корректно, в тег **head** надо добавить метатег **viewport**.

*HTML-код:*

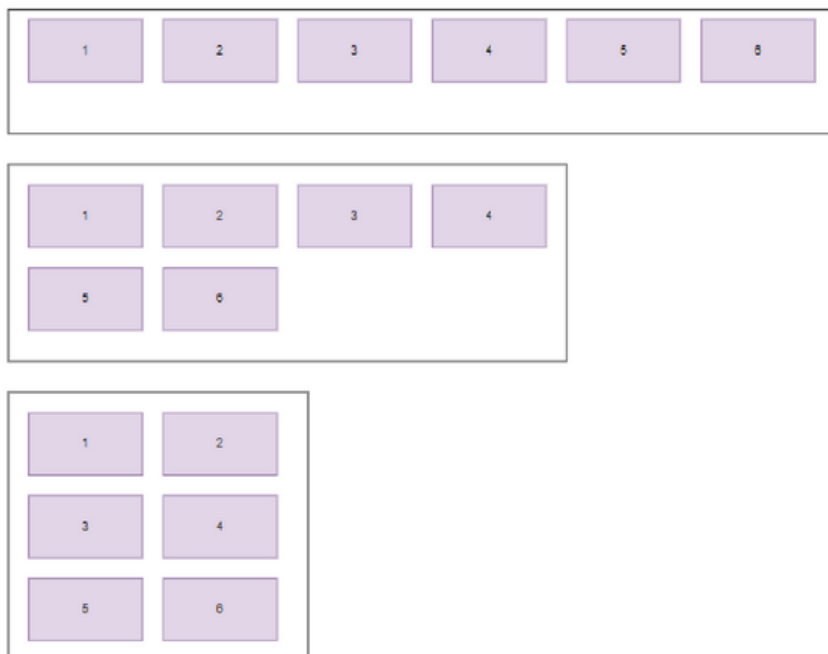
```
<head>
  ...
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1"
  >
  ...
</head>
```



# Задание 1

---

**Задание 1.** Расположить элементы страницы для разных размеров экрана, как показано на рисунке:



## Контейнеры

---

**Контейнеры** являются основным элементом макета в Bootstrap.

Контейнеры используются для размещения содержимого внутри них.

В Bootstrap поддерживается три вида контейнеров:

- **.container** — устанавливает **max-width** для каждой контрольной точки;
- **.container-fluid** — ширина содержимого равна **width: 100%** во всех контрольных точках;
- **.container-{breakpoint}** — ширина содержимого равна **width: 100%** до указанной контрольной точки.

# Контейнеры

Ширина контейнера в зависимости от размера экрана:

|                  | Сверхмалые<br><576px | Малые<br>≥576px | Средний<br>≥768px | Большие<br>≥992px | Очень<br>большой<br>≥1200px | Очень<br>очень<br>большой<br>≥1400 |
|------------------|----------------------|-----------------|-------------------|-------------------|-----------------------------|------------------------------------|
| .container       | 100%                 | 540px           | 720px             | 960px             | 1140px                      | 1320px                             |
| .container-sm    | 100%                 | 540px           | 720px             | 960px             | 1140px                      | 1320px                             |
| .container-md    | 100%                 | 100%            | 720px             | 960px             | 1140px                      | 1320px                             |
| .container-lg    | 100%                 | 100%            | 100%              | 960px             | 1140px                      | 1320px                             |
| .container-xl    | 100%                 | 100%            | 100%              | 100%              | 1140px                      | 1320px                             |
| .container-xxl   | 100%                 | 100%            | 100%              | 100%              | 100%                        | 1320px                             |
| .container-fluid | 100%                 | 100%            | 100%              | 100%              | 100%                        | 100%                               |

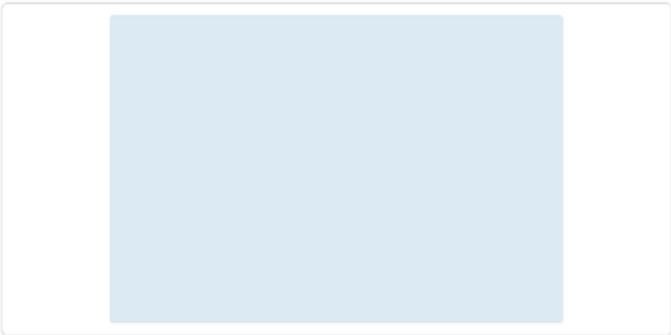
## Контейнеры, класс .container

Класс **.container** по умолчанию является адаптивным контейнером фиксированной ширины, что означает, что его максимальная ширина изменяется в каждой контрольной точке (класс **.bg-light** добавлен, чтобы установить фон контейнера).

HTML-код:

```
<div class="container bg-light">
  <!--содержимое контейнера -->
</div>
```

Результат:



# Адаптивные контейнеры

---

Адаптивные контейнеры позволяют указать класс шириной 100% до достижения указанной контрольной точки, после чего применяется **max-width** для каждой из более высоких контрольных точек.

Например, **.container-sm** имеет 100% ширину до тех пор, пока не будет достигнута контрольная точка **sm**, где он будет масштабироваться с помощью **md**, **lg**, **xl** и **xxl**.

# Адаптивные контейнеры

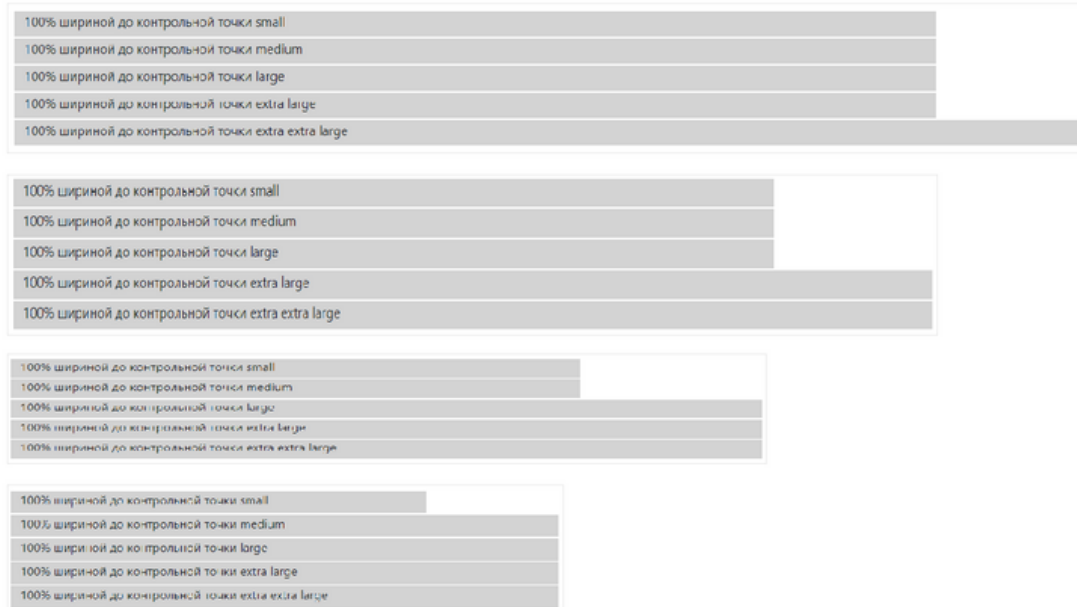
---

HTML-код:

```
<div class="container-sm bg-light">  
    100% шириной до контрольной точки small  
</div>  
<div class="container-md bg-light">  
    100% шириной до контрольной точки medium  
</div>  
<div class="container-lg bg-light">  
    100% шириной до контрольной точки large  
</div>  
<div class="container-xl bg-light">  
    100% шириной до контрольной точки extra large  
</div>  
<div class="container-xxl bg-light">  
    100% шириной до контрольной точки extra extra large  
</div>
```

# Адаптивные контейнеры

Результат:



## Контейнеры, `.container-fluid`

Класс **`.container-fluid`** используется для создания контейнера, охватывающего всю ширину области просмотра для любого размера экрана.

HTML-код:

```
<div class="container bg-light">  
  .container  
</div>  
<div class="container-fluid bg-light">  
  .container-fluid  
</div>
```

Результат:



# Макет Bootstrap

---

Макеты Bootstrap реализованы на архитектуре flexbox.

Макет Bootstrap обычно имеет следующую структуру:

- контейнер;
- строка;
- столбец.

## Макет Bootstrap

---

**Пример.** Включим в контейнер блоки, объединенные сначала в строки, а затем в столбцы.

HTML-код:

```
<div class="container">
  <div class="row">
    <div class="col">Строка 1, столбец 1</div>
    <div class="col">Строка 1, столбец 2</div>
    <div class="col">Строка 1, столбец 3</div>
  </div>
  <div class="row ">
    <div class="col">Строка 2, столбец 1</div>
    <div class="col">Строка 2, столбец 2</div>
    <div class="col">Строка 2, столбец 3</div>
  </div>
  <div class="row">
    <div class="col">Строка 3, столбец 1</div>
    <div class="col">Строка 3, столбец 2</div>
    <div class="col">Строка 3, столбец 3</div>
  </div>
</div>
```



# Макет Bootstrap

---

Добавим CSS стили, чтобы установить границу и фон блоков.

CSS-код:

```
.col {  
    background: lightgrey;  
    border: solid thin grey;  
}  
  
.row {  
    height: 50px;  
    border: solid thin red;  
}
```

# Макет Bootstrap

---

*Результат:*

|                     |                     |                     |
|---------------------|---------------------|---------------------|
| Строка 1, столбец 1 | Строка 1, столбец 2 | Строка 1, столбец 3 |
| Строка 2, столбец 1 | Строка 2, столбец 2 | Строка 2, столбец 3 |
| Строка 3, столбец 1 | Строка 3, столбец 2 | Строка 3, столбец 3 |

# Макет Bootstrap

---

Добавим CSS стили, чтобы установить границу и фон блоков.

CSS-код:

```
.col {  
    background: lightgrey;  
    border: solid thin grey;  
}  
  
.row {  
    height: 50px;  
    border: solid thin red;  
}
```

## Вертикальное выравнивание

---

В Bootstrap определены следующие классы для вертикального выравнивания столбцов в строке:

- **.align-items-start** – прижимает столбцы к «верху» строки;
- **.align-items-center** – располагает столбцы по центру строки;
- **.align-items-end** – прижимает столбцы к «низу» строки.

# Вертикальное выравнивание

---

В нашем примере установим различные виды вертикального выравнивания содержимого строки.

HTML-код:

```
<div class="container">
  <div class="row align-items-start">
    ...
  </div>
  <div class="row align-items-center">
    <div class="col">
      ...
    </div>
  <div class="row align-items-end">
    ...
  </div>
</div>
```

# Вертикальное выравнивание

---

Результат:

|                     |                     |                     |
|---------------------|---------------------|---------------------|
| Строка 1, столбец 1 | Строка 1, столбец 2 | Строка 1, столбец 3 |
| Строка 2, столбец 1 | Строка 2, столбец 2 | Строка 2, столбец 3 |
| Строка 3, столбец 1 | Строка 3, столбец 2 | Строка 3, столбец 3 |

# Вертикальное выравнивание

---

Для вертикального выравнивания столбцов внутри строки используются следующие классы:

- **.align-self-start** – прижимает столбцы к «верху» строки;
- **.align-self-center** – располагает столбцы по центру строки;
- **.align-self-end** – прижимает столбцы к «низу» строки.

## Вертикальное выравнивание

---

В нашем примере установим различные виды вертикального выравнивания столбцов.

HTML-код:

```
<div class="container">
  <div class="row">
    <div class="col align-self-start">
      Строка 1, столбец 1
    </div>
    <div class="col align-self-center">
      Строка 1, столбец 2
    </div>
    <div class="col align-self-end">
      Строка 1, столбец 3
    </div>
  </div>
</div>
```

# Вертикальное выравнивание

---

Результат:

|                     |                     |                     |
|---------------------|---------------------|---------------------|
| Строка 1, столбец 1 | Строка 1, столбец 2 | Строка 1, столбец 3 |
|---------------------|---------------------|---------------------|

# Горизонтальное выравнивание

---

В Bootstrap определены следующие классы для горизонтального выравнивания столбцов в строке:

**.justify-content-start** – столбцы сдвинуты в начало строки;

**.justify-content-center** – столбцы размещены по центру строки;

**.justify-content-end** – столбцы сдвинуты в конец строки;

**.justify-content-around** – столбцы равномерно распределены по строке, вокруг каждого столбца одинаковое пространство;

**.justify-content-between** – столбцы равномерно распределены по строке, первый элемент находится в начале строки, последний элемент в конце строки;

**.justify-content-evenly** – столбцы распределены таким образом, чтобы расстояние между любыми двумя элементами (и расстояние до границ) было одинаковым.



# Горизонтальное выравнивание

---

В нашем примере установим различные установим различные типы горизонтального выравнивания.

HTML-код:

```
<div class="container">
  <div class="row justify-content-start">
    <div class="col-4">Строка 1, столбец 1</div>
    <div class="col-4">Строка 1, столбец 2</div>
  </div>
  <div class="row justify-content-center">... </div>
  <div class="row justify-content-end"> ... </div>
  <div class="row justify-content-around"> ... </div>
  <div class="row justify-content-between"> ... </div>
  <div class="row justify-content-evenly"> ... </div>
</div>
```

# Горизонтальное выравнивание

---

HTML-код:

```
<div class="container">
  <div class="row justify-content-start">
    <div class="col-4">Строка 1, столбец 1</div>
    <div class="col-4">Строка 1, столбец 2</div>
  </div>
  <div class="row justify-content-center">... </div>
  <div class="row justify-content-end"> ... </div>
  <div class="row justify-content-around"> ... </div>
  <div class="row justify-content-between"> ... </div>
  <div class="row justify-content-evenly"> ... </div>
</div>
```

# Горизонтальное выравнивание

Результат:

|                     |                     |                     |
|---------------------|---------------------|---------------------|
| Строка 1, столбец 1 | Строка 1, столбец 2 |                     |
|                     | Строка 2, столбец 1 | Строка 2, столбец 2 |
|                     | Строка 3, столбец 1 | Строка 3, столбец 2 |
|                     | Строка 4, столбец 1 | Строка 4, столбец 2 |
| Строка 5, столбец 1 |                     | Строка 5, столбец 2 |
|                     | Строка 6, столбец 1 | Строка 6, столбец 2 |

## Способ отображения

Классы для управления способом отображения элементами именуются в следующем формате:

**.d-значение** или

**.d-breakpoint-значение** для *sm*, *md*, *lg*, *xl* и *xxl*,

где *значение* – одно из *none*, *inline*, *inline-block*, *block*, *grid*, *table*, *table-cell*, *table-row*, *flex*, *inline-flex*.

# Способ отображения

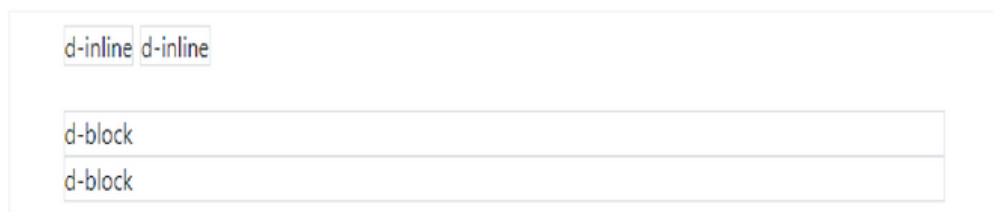
---

**Пример.** Покажем два **div** в строчном и блочном отображении.

HTML-код:

```
<div class="container">
  <div class="d-inline border">d-inline</div>
  <div class="d-inline border">d-inline</div>
</div>
<br>
<div class="container">
<span class="d-block border">d-block</span>
<span class="d-block border">d-block</span>
</div>
```

Результат:



# Способ отображения

---

В зависимости от способа отображения родительского элемента изменяется отображение его дочерних элементов.

# Способ отображения

---

**Пример.** В контейнер включим несколько элементов, будем указывать различные способы отображения контейнера и исследуем, как будут меняться отображение его дочерних элементов.

HTML-код:

```
<div class="container border d-значение">  
  <h5 class="bg-light">Ворота на запад</h5>  
  <p>192-метровая арка в Сент-Луисе (штат Миссури, США)</p>  
    
</div>
```

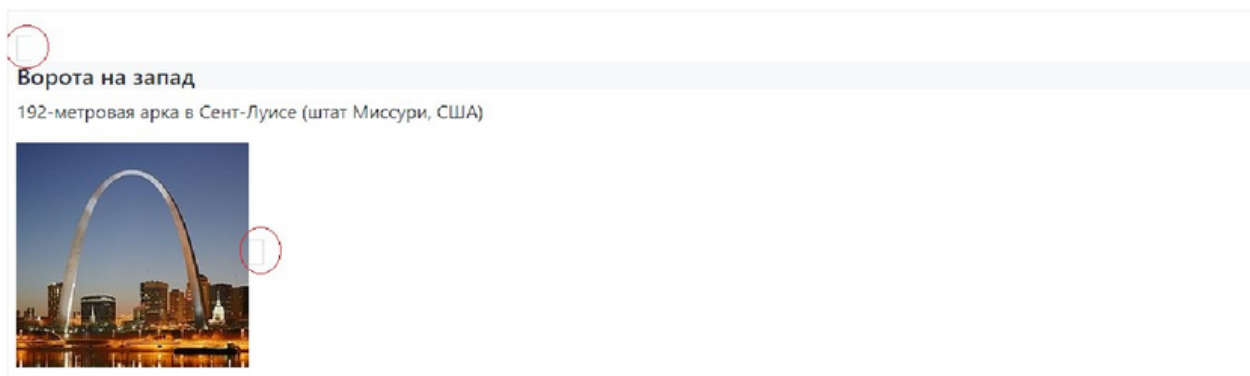
# Способ отображения

---

## d-none

Контейнер становится скрытым, в браузере не отображается.

## d-inline



У контейнера границы изменяются, содержимое занимает всю доступную ширину.

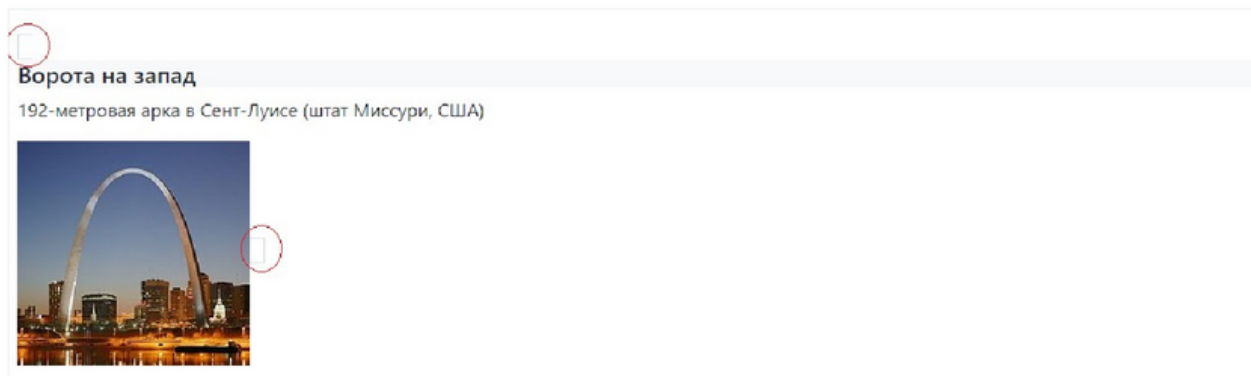
# Способ отображения

---

## d-none

Контейнер становится скрытым, в браузере не отображается.

## d-inline

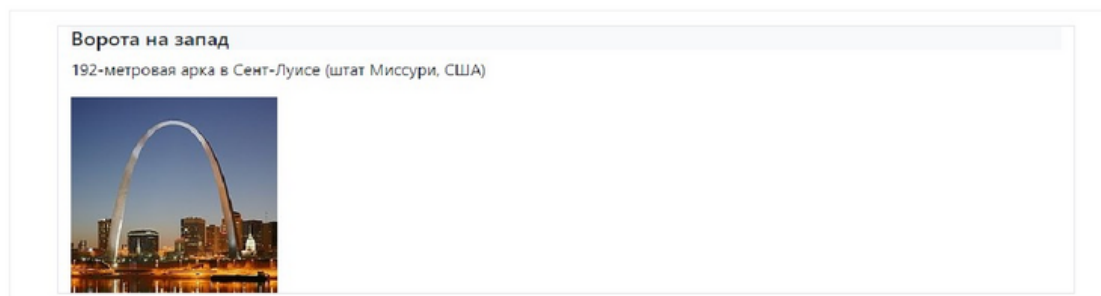


У контейнера границы изменяются, содержимое занимает всю доступную ширину.

# Способ отображения

---

## d-block



Контейнер выравнивается по центру, его ширина определяется размером экрана.

# Способ отображения

---

## d-inline-block

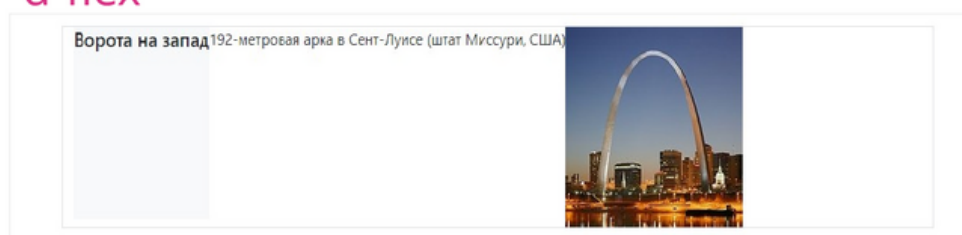


Контейнер выравнен по левой границе, его ширина определяется размером экрана.

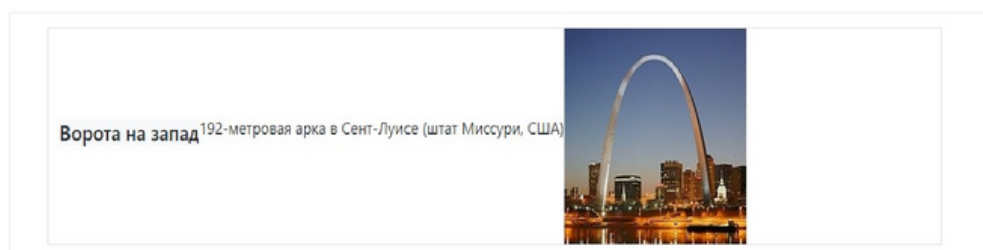
# Способ отображения

---

## d-flex



Контейнер выравнен по центру границе, его ширина определяется размером экрана, его содержимое перестраивается в одну строку. Можно использовать свойства flex-элементов, например, в контейнере установить вертикальное выравнивание (**align-items-center**).





# Способ отображения

## d-grid

Ворота на запад

192-метровая арка в Сент-Луисе (штат Миссури, США)



Контейнер выравнен по центру границе, его ширина определяется размером экрана. Можно использовать свойства grid-элементов, например, в контейнере установить горизонтальное выравнивание (**justify-content-end**).

Ворота на запад

192-метровая арка в Сент-Луисе (штат Миссури, США)



## Задание 2

**Задание 2.** Добавить классы в контейнер, чтобы страница выглядела как на первом рисунке, а затем на втором.

```
<div class="container border">
```

Ворота  
на  
запад

192-метровая арка в  
Сент-Луисе (штат  
Миссури, США)



Ворота  
на  
запад

192-метровая арка в  
Сент-Луисе (штат  
Миссури, США)

Ворота на запад

192-метровая арка в Сент-Луисе (штат Миссури, США)



Ворота на запад

192-метровая арка в Сент-Луисе (штат Миссури, США)

# Утилиты Bootstrap

---

**Утилиты** в Bootstrap — набор готовых классов, которые позволяют изменить или дополнить существующий стиль элементов на странице.

Каждая утилита в Bootstrap — это атомарный класс. Такой класс устанавливает только один стиль для элемента, например:

- цвет текста;
- границы;
- фон;
- выравнивание;
- тип отображения;
- и т.д.

## Утилиты Bootstrap

---

Пример атомарного класса:

```
.text-center {  
  text-align: center;  
}
```

Класс **.text-center** отвечает за один стиль — центрирование текста.

В Bootstrap есть соответствующая утилита – готовый для использования класс **.text-center**, стиль которого описывать не нужно.

Из таких «кирпичиков» можно строить различные дизайны страниц, создавать свои уникальные элементы в Bootstrap, изменять дизайн уже готовых компонентов

# Оформление текста, цвет

---

Для обозначения классов, с помощью которых устанавливается цвет текста, используется формат описания класса:

## **.text-цвет**

где **цвет** – это одно из значений: `text-primary`, `text-primary-emphasis`, `text-secondary`, `text-secondary-emphasis`, `text-success`, `text-success-emphasis`, `text-danger`, `text-danger-emphasis`, `text-warning`, `text-warning-emphasis`, `text-info`, `text-info-emphasis`, `text-light`, `text-light-emphasis`, `text-dark`, `text-dark-emphasis`, `text-body`, `text-body-emphasis`, `text-body-secondary`, `text-body-tertiary`, `text-black`, `text-white`, `text-black-50`, `text-white-50`.

# Оформление текста, цвет

---

Примеры цветов:

`.text-primary`

`.text-primary-emphasis`

`.text-secondary`

`.text-secondary-emphasis`

`.text-success`

`.text-success-emphasis`

`.text-danger`

`.text-danger-emphasis`

# Оформление, выравнивание

---

Классы для установки выравнивания текста называются по следующему формату:

**.text[-breakpoint]-атрибут,**

где

**breakpoint = sm | md | lg | xl | xll.**

**атрибут** принимает одно из следующих значений:

**start** – выравнивание по левой границе;

**center** – выравнивание по центру;

**end** – выравнивание по правой границе.

## Оформление текста, выделение

---

Атрибуты стиля `font-weight` в Bootstrap обозначаются так:

**.fw-атрибут,**

где **атрибут** – это одно из значений:

**bold** – текст жирным шрифтом;

**bolder** – текст жирнее (по сравнению с родительским элементом);

**semibold** – текст, выделенный полужирным шрифтом;

**medium** – текст средней жирности;

**normal** – обычный текст;

**light** – полупрозрачный текст;

**lighter** – текст прозрачнее(относительно родительского элемента).

# Оформление текста, выделение

---

Атрибуты стиля `font - style` сокращаются так

**.fst-атрибут,**

где **атрибут** – это одно из значений:

***italic*** – курсивный текст;

**normal** –обычный текст.

# Оформление текста, выделение

---

Атрибуты стиля `text-decoration` обозначаются так:

**.text-decoration-атрибут,**

где **атрибут** – это одно из значений:

**underline** – подчеркнутый текст;

**~~line-through~~** – зачеркнутый текст;

**none** – обычный текст (без подчеркивания и зачеркивания).

# Оформление текста, выделение

---

Примеры выделения текста:

**Текст жирным шрифтом.**

Текст, выделенный полужирным шрифтом.

Текст средней жирности.

Обычный текст.

Полупрозрачный текст.

*Курсив.*

Текст с подчеркиванием.

[Ссылка без подчеркивания.](#)

## Границы элементов

---

Классы для установки границ элемента толщиной в 1 пиксель называются по следующему формату:

**.border[-граница],**

для удаления части границы используется формат:

**.border border[-граница]-0,**

где **граница** – это одно из значений:

**top** – верхняя граница;

**end** – правая граница;

**bottom** – нижняя граница;

**start** – левая граница.



# Границы элементов, толщина

---

Для того, чтобы установить толщину границы используется формат:

**border[-граница] border-размер,**

где **размер** – толщина границы в пикселях от 1 до 5.

## Границы элементов, толщина

---

Пример.

```
<div class="col border border-2">
```

Элемент 1

Элемент 1

```
</div>
```

```
<div class="col border-top border-3">
```

Элемент 2

Элемент 2

```
</div>
```

```
<div class="col border-bottom border-4">
```

Элемент 3

Элемент 3

```
</div>
```

```
<div class="col border border-end-0">
```

Элемент 4

Элемент 4

```
</div>
```

```
<div class="col border-start border-1 border-top border-4">
```

Элемент 5

Элемент 5

```
</div>
```

## Границы элементов, цвет

---

Для того, чтобы установить цвет границы используется формат:

**border[-граница] border-цвет,**

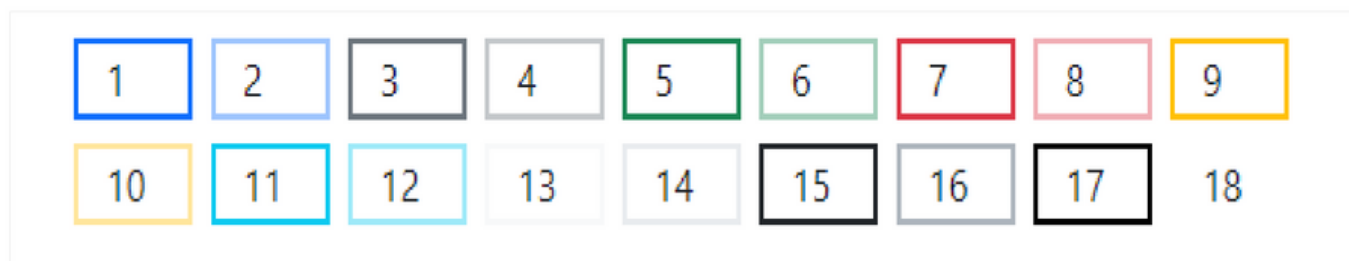
где **цвет** – это одно из значений:

primary, primary-subtle, secondary, secondary-subtle, success, success-subtle, danger, danger-subtle, warning, warning-subtle, border-info, border-info-subtle, light, light-subtle, dark, dark-subtle, black, white.

## Границы элементов, цвет

---

Примеры цветов границ:



# Границы элементов, прозрачность

---

Для того чтобы установить степень прозрачности границы используется формат:

**border[-граница] border-цвет border-opacity-процент**

где **процент** – это значение от 0 до 100 (0 – полностью прозрачная граница, 100 – полностью непрозрачная).



# Границы элементов, скругление

---

Для того чтобы установить скругленные углы элемента используется формат:

**border rounded[-граница]-значение,**

где **значение** – это одно из:

**1, 2, 3, 4, 5** – коэффициент скругления;

**circle** – граница в виде эллипса (круга);

**pill** – закругленные углы прямоугольника.

# Границы элементов, скругление

---

```
<div class="col border border-2 rounded-2">  
  2  
</div>  
<div class="col border border-2 rounded-start-3">  
  3  
</div>  
<div class="col border border-2 rounded-end-pill">  
  end-pill  
</div>  
<div class="col border border-2 rounded-top-circle">  
  top-circle  
</div>  
<div class="col border border-2 rounded-circle">  
  circle  
</div>  
<div class="col border border-2 rounded-pill">  
  pill  
</div>
```



The image shows seven visual examples of the classes defined in the code above, each in a light gray box. The examples are: '2' (a simple rectangle), '3' (a rectangle with rounded start corners), 'end-pill' (a rectangle with rounded end corners), 'top-circle' (a rectangle with a circular top corner), 'circle' (a fully rounded rectangle), and 'pill' (a rectangle with rounded corners and a thin border).

## Фон элементов

---

Для того чтобы установить фоновый цвет элемента используется формат:

**.bg-цвет**

где **цвет** – это одно из значений:

primary, primary-subtle, secondary, secondary-subtle, success, success-subtle, danger, danger-subtle, warning, warning-subtle, info, info-subtle, light, light-subtle, dark, dark-subtle, body-secondary, body-tertiary, body, black, white, transparent.

# Фон элементов

---

Примеры использования цвета фона:

`.bg-info`

`.bg-info-subtle`

`.bg-light`

`.bg-light-subtle`

`.bg-warning`

## Отступы

---

Классы отступов называются по следующему формату:

**`property[sides] - size`,**

или

**`property[sides] - breakpoint - size`,**

где

**`breakpoint = sm | md | lg | xl | xll`.**

**`property`** – это одно из значений:

**`m`** - задает **`margin`**;

**`p`** - задает **`padding`**;

**`g`** - задает расстояния между элементами сеток при перестроении.

# Отступы

---

`sides` – это одно из значений:

`t` - задает `margin-top` или `padding-top`;

`b` - задает `margin-bottom` или `padding-bottom`;

`l` - задает `margin-left` или `padding-left`;

`r` - задает `margin-right` или `padding-right`;

`x` - задает и `*-left` и `*-right`;

`y` - задает и `*-top` и `*-bottom`;

`пустое_значение` - задает `margin` или `padding` для всех 4-х сторон элемента.

# Отступы

---

`size` – это одно из значений (`$spacer = 1rem`):

`0` - удаляет `margin` или `padding` назначая его равным `0`;

`1` - (по умолчанию) устанавливает `margin` или `padding` к значению, равному `$spacer * 0.25`;

`2` - устанавливает `margin` или `padding` в `$spacer * 0.5`;

`3` - устанавливает `margin` или `padding` в `$spacer`;

`4` - устанавливает `margin` или `padding` в `$spacer * 1.5`;

`5` - устанавливает `margin` или `padding` в `$spacer * 3`;

`auto` - устанавливает `margin` как `auto`.



# Отступы

**Пример.** Создадим сетку из двух строк и трех столбцов.

HTML-код:

```
<div class="container">
  <div class="row border">
    <div class="col border">Строка 1, столбец 1</div>
    <div class="col border">Строка 1, столбец 2</div>
    <div class="col border">Строка 1, столбец 3</div>
  </div>
  <div class="row border">
    <div class="col border">Строка 2, столбец 1</div>
    <div class="col border">Строка 2, столбец 2</div>
    <div class="col border">Строка 2, столбец 3</div>
  </div>
</div>
```

Результат:

|                     |                     |                     |
|---------------------|---------------------|---------------------|
| Строка 1, столбец 1 | Строка 1, столбец 2 | Строка 1, столбец 3 |
| Строка 2, столбец 1 | Строка 2, столбец 2 | Строка 2, столбец 3 |

# Отступы

Установим внутренний отступ для строк.

```
<div class="container">
  <div class="row p-2 border">
    <div class="col border">Строка 1, столбец 1</div>
    <div class="col border">Строка 1, столбец 2</div>
    <div class="col border">Строка 1, столбец 3</div>
  </div>
  <div class="row p-2 border">
    <div class="col border">Строка 2, столбец 1</div>
    <div class="col border">Строка 2, столбец 2</div>
    <div class="col border">Строка 2, столбец 3</div>
  </div>
</div>
```

Результат:

|                     |                     |                     |
|---------------------|---------------------|---------------------|
| Строка 1, столбец 1 | Строка 1, столбец 2 | Строка 1, столбец 3 |
| Строка 2, столбец 1 | Строка 2, столбец 2 | Строка 2, столбец 3 |

# Отступы

Установим внутренний отступ для столбцов:

```
<div class="container">
  <div class="row p-2 border">
    <div class="col p-2 border">Строка 1, столбец 1</div>
    <div class="col p-2 border">Строка 1, столбец 2</div>
    <div class="col p-2 border">Строка 1, столбец 3</div>
  </div>
  <div class="row p-2 border">
    <div class="col p-2 border">Строка 2, столбец 1</div>
    <div class="col p-2 border">Строка 2, столбец 2</div>
    <div class="col p-2 border">Строка 2, столбец 3</div>
  </div>
</div>
```

Результат:

|                     |                     |                     |
|---------------------|---------------------|---------------------|
| Строка 1, столбец 1 | Строка 1, столбец 2 | Строка 1, столбец 3 |
| Строка 2, столбец 1 | Строка 2, столбец 2 | Строка 2, столбец 3 |

# Отступы

Установим внешний отступ для столбцов, для размера экрана больше, чем md.

```
<div class="container">
  <div class="row p-2 border">
    <div class="col p-2 m-md-2 border">
      Строка 1, столбец 1
    </div>
    ...
  </div>
  <div class="row p-2 border">
    <div class="col p-2 m-md-2 border">
      Строка 2, столбец 1
    </div>
    ...
  </div>
</div>
```

|                     |                     |                     |
|---------------------|---------------------|---------------------|
| Строка 1, столбец 1 | Строка 1, столбец 2 | Строка 1, столбец 3 |
| Строка 2, столбец 1 | Строка 2, столбец 2 | Строка 2, столбец 3 |

|                     |                     |                     |
|---------------------|---------------------|---------------------|
| Строка 1, столбец 1 | Строка 1, столбец 2 | Строка 1, столбец 3 |
| Строка 2, столбец 1 | Строка 2, столбец 2 | Строка 2, столбец 3 |

# Отступы

Установим внешний отступ для столбцов, для размера экрана больше, чем *md*.

```
<div class="container">
  <div class="row p-2 border">
    <div class="col p-2 m-md-2 border">
      Строка 1, столбец 1
    </div>
    ...
  </div>
  <div class="row p-2 border">
    <div class="col p-2 m-md-2 border">
      Строка 2, столбец 1
    </div>
    ...
  </div>
</div>
```

|                     |                     |                     |
|---------------------|---------------------|---------------------|
| Строка 1, столбец 1 | Строка 1, столбец 2 | Строка 1, столбец 3 |
| Строка 2, столбец 1 | Строка 2, столбец 2 | Строка 2, столбец 3 |

|                     |                     |                     |
|---------------------|---------------------|---------------------|
| Строка 1, столбец 1 | Строка 1, столбец 2 | Строка 1, столбец 3 |
| Строка 2, столбец 1 | Строка 2, столбец 2 | Строка 2, столбец 3 |

## Задание 3

**Задание 3.** Добавить классы в код, чтобы страница выглядела, как на рисунке.

```
<div class="container">
  <div class="row p-2 border">
    <div class="col p-2 m-md-2 border">
      Строка 1, столбец 1
    </div>
    <div class="col p-2 m-md-2 border">
      Строка 1, столбец 2
    </div>
  </div>
  <div class="row p-2 border">
    <div class="col p-2 m-md-2 border">
      Строка 2, столбец 1
    </div>
    <div class="col p-2 m-md-2 border">
      Строка 2, столбец 2
    </div>
  </div>
</div>
```

# Задание 3

---

**Задание 3.** Добавить классы в код, чтобы страница выглядела, как на рисунке.

|                     |                     |
|---------------------|---------------------|
| Строка 1, столбец 1 | Строка 1, столбец 2 |
| Строка 2, столбец 1 | Строка 2, столбец 2 |

|                     |
|---------------------|
| Строка 1, столбец 1 |
| Строка 1, столбец 2 |
| Строка 2, столбец 1 |
| Строка 2, столбец 2 |

## Размеры элементов

---

Классы для управления способом отображения элементами именуются в следующем формате:

**.w-значение** для обозначения ширины

**.h-значение** для обозначения высоты,

где **значение** - одно из **25%**, **50%**, **75%**, **100%** и **auto**.

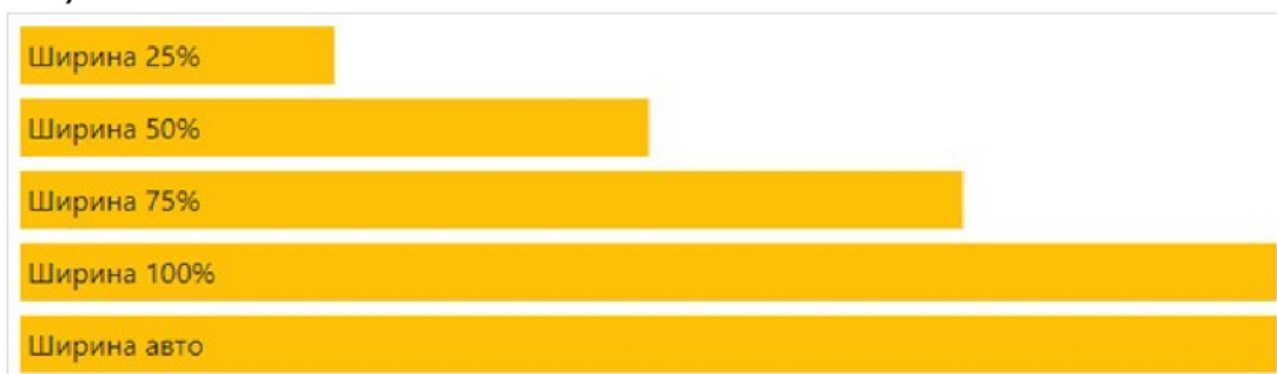
# Размеры элементов

---

HTML-код:

```
<div class="container">  
  <div class="w-25 my-2 p-1 bg-warning">Ширина 25%</div>  
  <div class="w-50 my-2 p-1 bg-warning">Ширина 50%</div>  
  <div class="w-75 my-2 p-1 bg-warning">Ширина 75%</div>  
  <div class="w-100 my-2 p-1 bg-warning">Ширина 100%</div>  
  <div class="w-auto my-2 p-1 bg-warning">Ширина авто</div>  
</div>
```

Результат:



# Размеры элементов

---

HTML-код:

```
<div class="container bg-light d-flex" style="height:100px">  
  <div class="h-25 mx-1 p-1 bg-warning" >Высота 25%</div>  
  <div class="h-50 mx-1 p-1 bg-warning">Высота 50%</div>  
  <div class="h-75 mx-1 p-1 bg-warning">Высота 75%</div>  
  <div class="h-100 mx-1 p-1 bg-warning">Высота 100%</div>  
  <div class="h-auto mx-1 p-1 bg-warning">Высота авто</div>  
</div>
```

Результат:





# Оформление рисунков

---

Класс **.img-fluid** позволяет сделать изображение масштабируемым на разных устройствах. Этот класс применяется к изображению стили **max-width: 100%** и **height: auto**. При использовании этого класса изображение будет масштабироваться в соответствии с родительским элементом.

**Пример.** Поместим рисунок в контейнер и сделаем его адаптивным.

HTML-код:

```
<div class="container">  
    
</div>
```

## Выравнивание рисунков

---

Классы для горизонтального выравнивания изображений называются по следующему формату:

**.float-атрибут** или **.float-breakpoint-атрибут**,

где **breakpoint = sm | md | lg | xl | xll**.

**атрибут** – это одно из значений:

**start** – выравнивание по левой границе;

**end** – выравнивание по правой границе;

**none** – отмена обтекания.

Чтобы использовать горизонтальное выравнивание необходимо для изображений установить отображение **.d-block** или другое с подобными свойствами.



# Выравнивание рисунков

---

**Пример.** В контейнере выровняем рисунки по правой и левой границе.

HTML-код:

```
<div class="container">
  
  
</div>
```

Результат:



## Центрирование рисунков

---

Для рисунков в блочном отображении центрирование осуществляется с помощью класса **.mx-auto**.

При использовании строчного отображения (для рисунков это отображение установлено по умолчанию) центрирование задается классом **.text-center** в родительском элементе.

# Центрирование рисунков

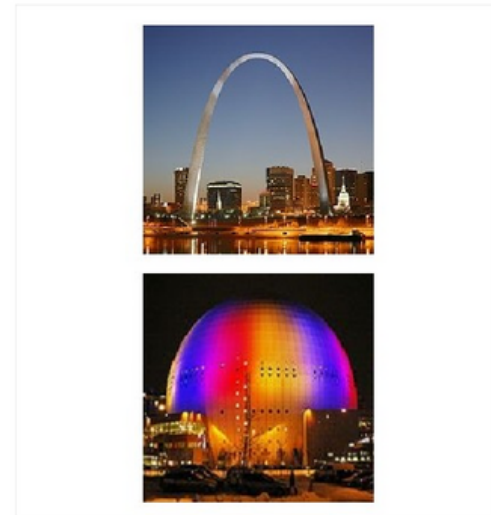
---

**Пример.** Выравниваем по центру рисунки с разным типом отображения.  
Для каждого контейнера установим внешний отступ.

HTML-код:

```
<div class="container m-3">
  
</div>

<div class="container m-3 text-center">
  
</div>
```



Спасибо за внимание!